

Projeto Refatoração

Arthur Carneiro

1 exercício

```
@PostMapping("/clientes")
public ResponseEntity cadastrar(@RequestBody Cliente cliente) {

    if (cliente.getNome() == null || cliente.getNome().isBlank()) {
        return ResponseEntity.badRequest()
            .body("Nome obrigatório");
    }

    if (clienteRepository.existsByEmail(cliente.getEmail())) {
        return ResponseEntity.badRequest()
            .body("Email já cadastrado");
    }

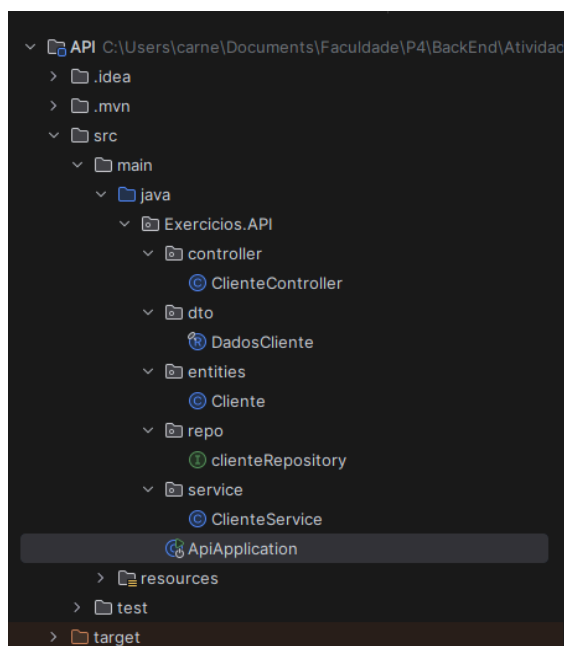
    cliente.setAtivo(true);
    cliente.setDataCadastro(LocalDateTime.now());

    Cliente salvo = clienteRepository.save(cliente);

    return ResponseEntity.ok(salvo);
}
```

Esse foi o código proposto para a refatoração. Apesar do código proposto funcionar, está tudo junto e misturado!

A primeira coisa que foi feito para ajudar foi separar em suas categorias!



CONTROLLER

```
DadosCliente.java  ApiApplication.java  ClienteController.java
1 package Exercicios.API.controller;
2
3 > import ...
11
12 @RestController
13 public class ClienteController {
14     @PostMapping("/clientes")
15     public ResponseEntity cadastrar(@RequestBody DadosCliente cliente) {
16         return ResponseEntity.ok(ClienteService.cadastrarCliente(cliente));
17     }
18 }
```

A diferença entre essa foto e a antiga, é que agora o controller está bem mais minúsculo apesar de fazer o mesmo trabalho!

DTO e SERVICE

```
DadosCliente.java  ApiApplication.java  ClienteController.java
1 package Exercicios.API.dto;
2
3 > import ...
6
7
8 public record DadosCliente(Object email, String nome) { 8 usages
9
10 @ >     public DadosCliente(Cliente cliente) { this(cliente.getEmail(), cliente.getNome()); }
13
14     public void setAtivo(boolean b){ 1 usage
15
16     public void setDataCadastro(LocalDateTime now){ 1 usage
17
18 }
```

```

@Service 2 usages Arthur Carneiro *
public class ClienteService {
    public static ResponseEntity cadastrarCliente(DadosCliente cliente) { 1 usage Arthur Carneiro *
        verificarCadastro(cliente);
        definirCliente(cliente);

        return clienteRepository.save(cliente);
    }

    public static ResponseEntity verificarCadastro(DadosCliente cliente) { 1 usage Arthur Carneiro
        if (cliente.nome() == null || cliente.nome().isBlank()) {
            return ResponseEntity.badRequest()
                .body("Nome obrigatório");
        }
        if (clienteRepository.existsByEmail(cliente.email())) {
            return ResponseEntity.badRequest()
                .body("Email já cadastrado");
        }
        return null;
    }

    public static void definirCliente(DadosCliente cliente) { 1 usage Arthur Carneiro
        cliente.setAtivo(true);
        cliente.setDataCadastro(LocalDate.now());
    }
}

```

Agora com DTO, vamos passar essas informações entre camadas. E a camada de Serviço está responsável pelas regras de negócio e validação. Comparado ao código antigo, essas atualizações são muito mais seguras por poderem proteger os dados recebidos para que cheguem de forma correta, além de que fica fácil a manutenção do código!